



sed & awk

NCSC101

Prof. Pranay Kumar Saha

August 25, 2025





What is sed?

- Stream Editor: reads input, applies script, writes output
- Pipeline-friendly, non-interactive
- This deck focuses on GNU sed (behavior can vary across BSD/BusyBox)



Basic Syntax

```
sed [OPTIONS] 'SCRIPT' [FILE ...]  
sed --help
```



Command-Line Options — Overview

Unless noted GNU-specific, these are widely available.

- Input/Output control (e.g., -n, -i, -s, -u, -z)
- Script loading (-e, -f)
- Regex mode (-E / -r)
- Diagnostics, POSIX mode, version/help



Command-Line Options —I/O and Control

Flag	Meaning
-n, --quiet, --silent	Suppress automatic printing of pattern space
-i[SUFFIX]	In-place edit (optional backup suffix)
-s	Treat files as separate rather than as one continuous stream
-u	Unbuffered output (minimize buffering)
-z	Input is NUL-separated; newline is data



Examples — I/O and Control Options

-n : suppress auto-print, show only explicit prints

```
echo -e "a  
b" | sed -n 's/a/A/p'
```

input: a b => output: A

-i : in-place edit with backup

```
echo "foo" > tmp.txt  
sed -i.bak 's/foo/bar/' tmp.txt
```

input: foo => file content: bar

-s : treat multiple files separately

```
sed -n -s '1p' file1 file2
```

prints first line of each file, not combined



Examples — I/O and Control Options

-u : unbuffered output (useful in pipelines)

```
tail -f log | sed -u 's/error/ERROR/g'
```

NUL-separated input

```
sed -z 's/a/A/g; S/NUL/ / g'
```



Treating Files Separately: The -s Option

- The **-s** (**--separate**) option tells **sed** to treat each file as a distinct entity.
- This resets line numbers and address ranges (like **1** or **\$**) for each new file.



Treating Files Separately: The -s Option

- The -s (`--separate`) option tells `sed` to treat each file as a distinct entity.
- This resets line numbers and address ranges (like `1` or `$`) for each new file.

Example Files

file1.txt

Apple
Banana

file2.txt

Carrot
Date



Treating Files Separately: The -s Option

Without -s (Default)

Files are treated as one continuous stream. The command '1p' only prints the very first line of the combined input.

```
$ sed -n '1p' file1.txt file2.txt
# Output:
Apple
```

With -s (Separate)

The line counter is reset for each file. The command '1p' now prints the first line of every file.

```
$ sed -n -s '1p' file1.txt file2.txt
# Output:
Apple
Carrot
```



Command-Line Options —Scripting

Flag	Meaning
-e SCRIPT	Add script to the commands to be executed
-f FILE	Add commands from file
-E, -r	Use extended regular expressions (ERE)



Command-Line Options — Compliance, Info

Flag	Meaning
--posix	Strict POSIX mode (GNU)
--sandbox	Disable e.g. file writes/reads for safety (GNU)
--version	Show version
--help	Show help



Examples —CLI Options

Print only what you explicitly ask for (-n) and -e for multiple commands

```
sed -n -e '1,3p' -e 's/error/ERROR/p' app.log
```

In-place edit with backup suffix

```
sed -i.bak 's/DEBUG/INFO/g' *.conf
```

Extended regex (ERE)

```
sed -E 's/\b(cat|dog)s?\b/pet/g' animals.txt
```



POSIX Regex: BRE vs ERE (symbols)

Meaning	BRE (Basic)	ERE (Extended)
Start / end of line	<code>^, \$</code>	<code>^, \$</code>
Any char (not <code>\n</code>)	<code>.</code>	<code>.</code>
Char class / negated	<code>[abc], [^abc]</code>	<code>[abc], [^abc]</code>
Group (capturing)	<code>\(... \)</code>	<code>(...)</code>
Alternation	<code>\ </code>	<code> </code>
Zero or one	<code>\?</code>	<code>?</code>
One or more	<code>\+</code>	<code>+</code>
Repeat exactly / range	<code>\{n\}, \{n,m\}</code>	<code>{n}, {n,m}</code>
Word start / end or boundary	<code>\<, \></code>	<code>\b, \b</code>



Addressing

- **Line numbers:** 1, 5, \$ (last line)
- **Regex addresses:** `/pattern/` -> `/address/ { s/foo/bar/; p }` on matched lines: substitute, then print
- **Ranges:** 1,10, `/start/,/end/`
 - `1,10` -> `sed -n '1,10p' file`: Print first 10 lines (1 through 10)
 - `/start/,/end/` -> `sed -n '/^BEGIN/,/^END/p' file`: Print from the first “BEGIN” up to the next “END”
- **first~step:** 1~2 (every second line from 1)
- **Negation:** ! after an address/range to invert



Common Commands (Editors)

Command	Description
<code>s/RE/REPL/FLAGS</code>	Substitute (see next slides for flags)
<code>d</code>	Delete pattern space; start next cycle
<code>p</code>	Print pattern space
<code>a \\text</code>	Append text after current line
<code>i \\text</code>	Insert text before current line
<code>c \\text</code>	Change (replace matched lines with text)
<code>y/SET1/SET2/</code>	Transliterate characters



Append, Insert, and Change – Example

fruits.txt

```
apple  
banana  
cherry  
date  
elderberry
```

Append

```
sed '/^banana$/a\  
Note: Yellow Fruits  
' fruits.txt
```

Insert

```
sed '/^cherry$/i\  
-- stone fruits --  
' fruits.txt
```

Change

```
sed '/^date$/c\  
DATE  
' fruits.txt
```



Transliterate Example

```
sed 'y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/' fruits.txt
```



Chaining Commands: The Semicolon ;

- The semicolon ; is used to separate multiple `sed` commands within a single script expression.
- It allows you to apply several actions in sequence to the same line without needing multiple `-e` flags.



Chaining Commands: The Semicolon ;

- The semicolon ; is used to separate multiple `sed` commands within a single script expression.
- It allows you to apply several actions in sequence to the same line without needing multiple `-e` flags.

Goal: On lines with “ERROR”, replace “log” with “LOG” and then delete the line.



Chaining Commands: The Semicolon ;

Method 1: Using -e

Each command is in a separate expression. This is more verbose but can be clear.

```
$ sed -e '/ERROR/s/log/LOG/' \  
      -e '/ERROR/d' \  
      app.log
```

Method 2: Using ;

Both commands are chained inside one script. This is more compact and efficient.

```
$ sed '/ERROR/{s/log/LOG/;d}' \  
      app.log
```

Note: Grouping the commands in braces {} after the address /ERROR/ is a common and readable practice.



Flow & Space Commands

Command	Description
<code>=</code>	Print current line number
<code>h,H</code>	Copy/append to hold space
<code>g,G</code>	Replace/append from hold space
<code>x</code>	Exchange hold and pattern space
<code>n,N</code>	Read next line (N appends with newline)
<code>P</code>	Print up to first newline in pattern space
<code>D</code>	Delete up to first newline; restart cycle
<code>q[NUM]</code>	Quit (optionally with exit code)



Example: = print current line number

fruits.txt

```
apple  
banana  
cherry  
date  
elderberry
```

```
sed -n '=';p' fruits.txt
```

Output:

```
1  
apple  
2  
banana  
3  
cherry  
4  
date  
5  
elderberry  
6
```



The Core Concept: Two Buffers

`sed` has two temporary storage areas (buffers) to work with text:

1. Pattern Space

- The active workspace or "workbench".
- A line is read from input into this space.
- All commands (`s`, `d`, etc.) operate on it.
- It's printed to output at the end of the cycle.

2. Hold Space

- A secondary buffer or "storage drawer".
- It's empty by default.
- You can use it to save text from the pattern space and retrieve it later.
- Its contents persist across line cycles.



Moving Data: Hold & Get Commands

Saving to the Hold Space

- **h:** **hold** → Overwrite hold space with pattern space.
- **H:** **Hold** → Append pattern space to hold space.

Retrieving from the Hold Space

- **g:** **get** → Overwrite pattern space with hold space.
- **G:** **Get** → Append hold space to pattern space.
- **x:** **exchange** → Swap the contents of both spaces.



Moving Data: Hold & Get Commands

Example: Reverse every two lines

```
# Save the first line (h), read the next (n),  
# print it (p), get the saved line (g), print it (p).  
$ sed -n 'h;n;p;g;p' file .txt
```

Input

```
apple  
banana  
cherry  
date
```



Moving Data: Hold & Get Commands

Example: Reverse every two lines

```
# Save the first line (h), read the next (n),  
# print it (p), get the saved line (g), print it (p).  
$ sed -n 'h;n;p;g;p' file .txt
```

Input

```
apple  
banana  
cherry  
date
```

Output

```
banana  
apple  
date  
cherry
```



Line Management: n, N, P, D

These commands alter the normal line-by-line flow.

Reading the Next Line

- **n**: Print pattern space, then replace it with the **next** input line.
- **N**: Append the **Next** input line to the pattern space (with a '\n').

Partial Print/Delete (for multi-line pattern space)

- **P**: **P**rint the pattern space up to the first newline.
- **D**: **D**delete the pattern space up to the first newline; restart cycle.



Line Management: n, N, P, D

Example: Join a line with the one after it

```
# Find a line , append the Next (N),  
# then replace the embedded newline.  
$ sed '/ERROR/{N;s/\n/ -- /}' app.log
```



Flow Control: `q[NUM]` —quit (with optional exit)

```
sed '2q' nums.txt
```

```
# Quit after printing the first 2 lines
```



File I/O Commands

Command	Description
r FILE	Read file contents and insert after current line
w FILE	Write pattern space to file (append)



Example of Backreferences

Command

```
sed -E 's/([ ^ ]+) ([ ^ ]+)/\2, \1/' names.txt
```

- -E: use Extended Regular Expressions (ERE).
- s/.../.../: substitute the first match on each line.
- ([^]+): capture group 1 = one-or-more non-space chars (first name).
- (single space): requires exactly one space between names.
- ([^]+): capture group 2 = one-or-more non-space chars (last name).
- Replacement: \2, \1 → *Last, First* using backreferences.

Example

Input: Alice Smith

Output: Smith, Alice



Substitute Flags: Overview

Syntax: `s/<regex>/<replacement>/<flags>`

- Position: after the third delimiter
- Multiple flags can be combined, e.g., `gp`, `gI`, `3wfile`



Substitute Flags — Standard

Flag	Meaning
g	Global: replace all matches in pattern space
n (number)	Replace the n-th match only (e.g., <code>s/a/b/3</code>)
p	Print pattern space if a replacement was made
w FILE	Write pattern space to FILE if replacement made



sed printing modes

Command

```
sed 's/a/A/p' fruits.txt
```

Output

```
Apple  
Apple  
banana  
bAnana  
cherry  
date  
dAte  
elderberry
```

Command

```
sed -n 's/a/A/p' fruits.txt
```

Output

```
Apple  
bAnana  
dAte
```

Command

```
sed -n 's/a/A/;p' fruits.txt
```

Output

```
Apple  
banana  
cherry  
dAte  
elderberry
```



Further Reading

- `man sed`
- `info sed`
- GNU sed manual: <https://www.gnu.org/software/sed/manual/>



sed & awk

Thank You for Listening!